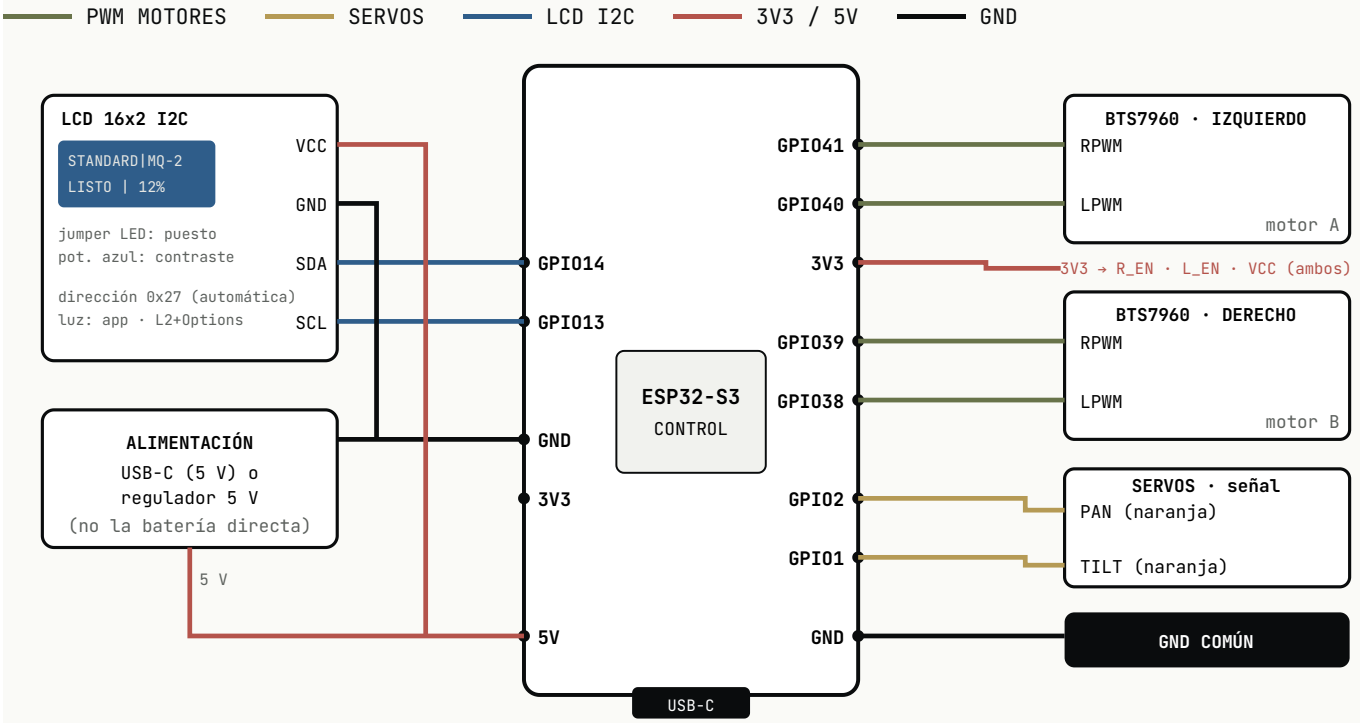
SHADOWBOT

A ESP32-S3 CONTROL · MOTORES, BRAZO Y LCD



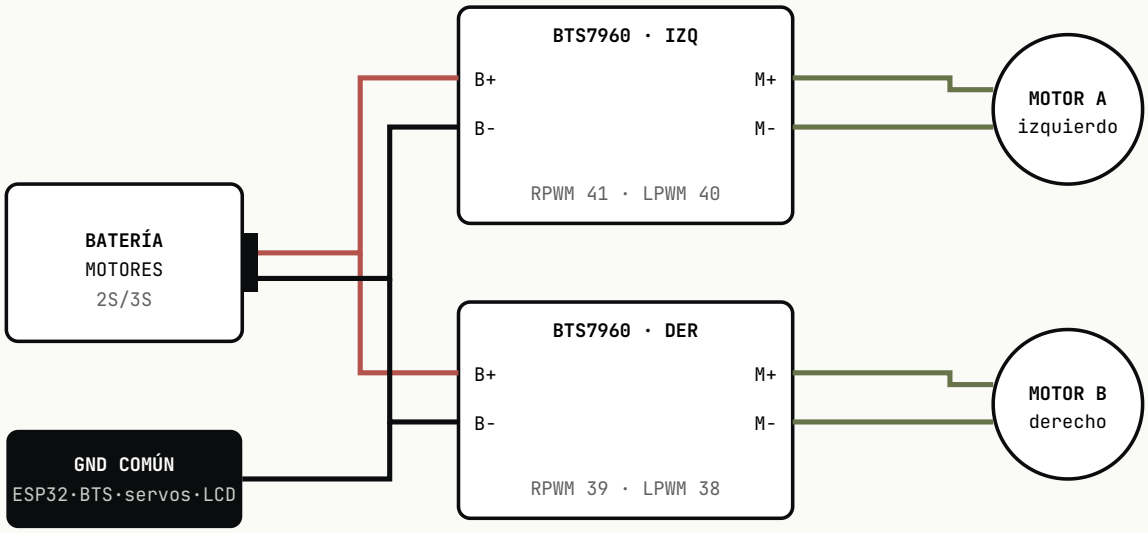
LCD: 4 pines seguidos 13·14·5V·GND (cables derechos). 3V3 de los BTS: pin derecho.

TABLA PIN POR PIN

PIN ESP32-S3	VA A	NOTA
GPIO41	BTS7960 IZQUIERDO · RPWM	PWM 20 kHz, 8 bits
GPIO40	BTS7960 IZQUIERDO · LPWM	PWM
GPIO39	BTS7960 DERECHO · RPWM	PWM
GPIO38	BTS7960 DERECHO · LPWM	PWM
GPIO2	Servo PAN · señal	50 Hz, 500–2400 µs
GPIO1	Servo TILT · señal	50 Hz · no usar 35–37 (PSRAM)
GPIO14	LCD 16x2 I2C · SDA	4 pines seguidos al final de la tira izquierda:
GPIO13	LCD 16x2 I2C · SCL	13 · 14 · 5V · GND, mismo orden que el módulo
3V3	BTS7960 (los dos) · VCC, R_EN, L_EN	lógica, no potencia
5V	USB-C o regulador 5 V · LCD VCC	alimenta al ESP32-S3 y al LCD
GND	BTS7960 · servos · batería · regulador · LCD	GND COMÚN obligatorio

CÓDIGO: CODIGOS SIN GC9A01\CONTROL\CONTROL.ino · placa "ESP32S3 Dev Module" · **PSRAM: Disabled** · librería ESP32Servo (el LCD no necesita librería). Para probar motores y servos por el Monitor Serie: PRUEBA_BTS_SERVOS (en esta misma placa).

B ALIMENTACIÓN Y ETAPA DE POTENCIA



ELEMENTO	CONEXIÓN
Batería motores	Solo a B+ / B− de cada BTS7960. Nunca a los pines del ESP32-S3.
BTS7960 lógica	VCC, R_EN y L_EN → 3V3 del ESP32-S3. GND lógica → GND del ESP32-S3.
Servos	Rojo → 5 V (de preferencia BEC/fuente externa) · café → GND común · naranja → GPIO2 (PAN) / GPIO1 (TILT).
ESP32-S3	5 V por USB-C o regulador. Comparte GND con todo lo demás.
LCD 16x2 I2C	VCC → 5 V · GND → GND común · SDA → GPIO14 · SCL → GPIO13 (4 pines seguidos: 13, 14, 5V, GND). Lo más seguro: un convertidor de nivel I2C 5 V ↔ 3.3 V entre el LCD y la ESP32-S3.
Motor al revés	No hay que recablear: app → Sistema → Conexiones → "Dirección de motores".

GND COMÚN. Batería de motores, BTS7960, servos y ESP32-S3 deben compartir tierra; sin eso el PWM no tiene referencia y los motores no responden o se mueven solos.

PINES LIBRES DE CONFLICTO. Ninguno de estos pines choca con USB (19–20), flash/PSRAM (26–37; en la N16R8 los 35–37 van a la PSRAM octal) ni arranque (0, 3, 45, 46). Los servos van en GPIO2 / GPIO1 y el LCD en GPIO14 / GPIO13. Cargue CONTROL con **`PSRAM: Disabled`**.

C ESP8266 (NODEMCU / WEMOS) · MQ-2 Y 4 LEDS

SALIDA LED · ANALÓGICO A0 · 5V / VIN · GND

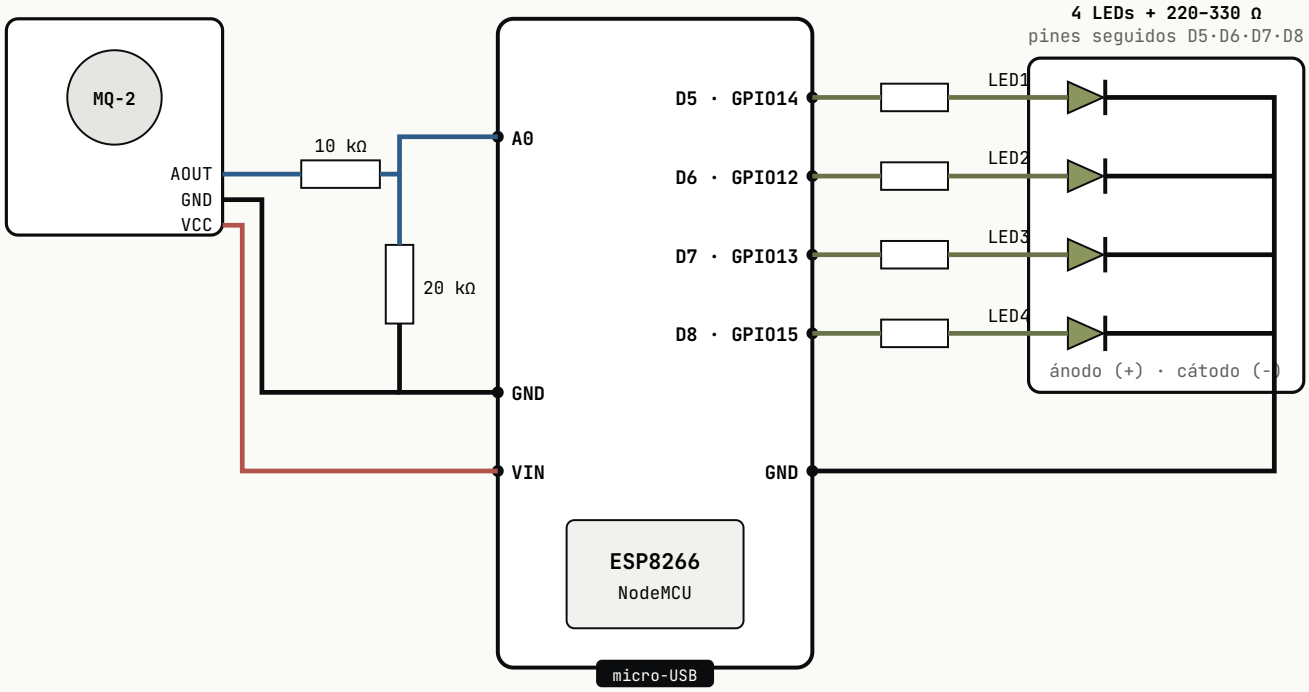


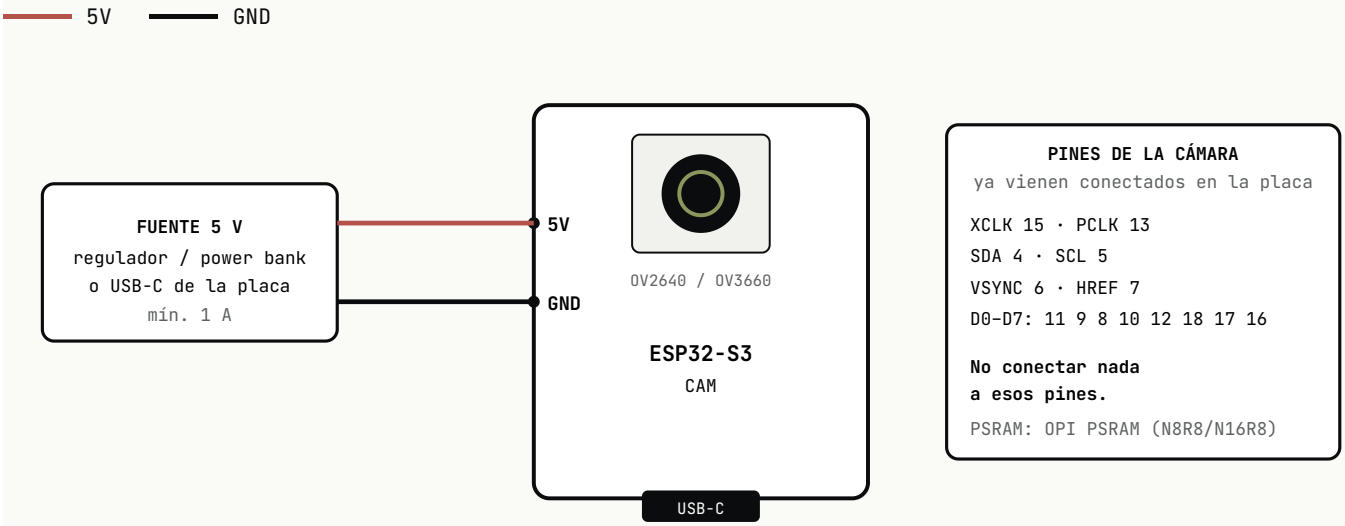
TABLA PIN POR PIN

PIN ESP8266	VA A	NOTA
A0	MQ-2 · AOUT (a través del divisor)	10 kΩ en serie, 20 kΩ a GND. Módulo ESP-12 suelto: 47 kΩ / 10 kΩ
VIN (5 V)	MQ-2 · VCC	el calentador del MQ-2 necesita 5 V
GND	MQ-2 · GND, cátodos de los 4 LEDs, divisor	
D5 · GPIO14	LED1 (con resistencia)	LED de CAMARA en la vinculación
D6 · GPIO12	LED2 (con resistencia)	LED de CONTROL
D7 · GPIO13	LED3 (con resistencia)	LED de SENSORES
D8 · GPIO15	LED4 (con resistencia)	LED de la Tapo / faros. Arranca bien: el LED a GND lo deja en LOW

LEDs SEGUIDOS: D5·D6·D7·D8 están juntos en la NodeMCU y en la D1 mini. **NO USAR** D3, D4 (deciden cómo arranca la placa) ni D0.

CÓDIGO: CODIGOS SIN GC9A01\SENSORES_8266\SENSORES_8266.ino · placa "NodeMCU 1.0 (ESP-12E Module)" o "LOLIN(WEMOS) D1 mini".

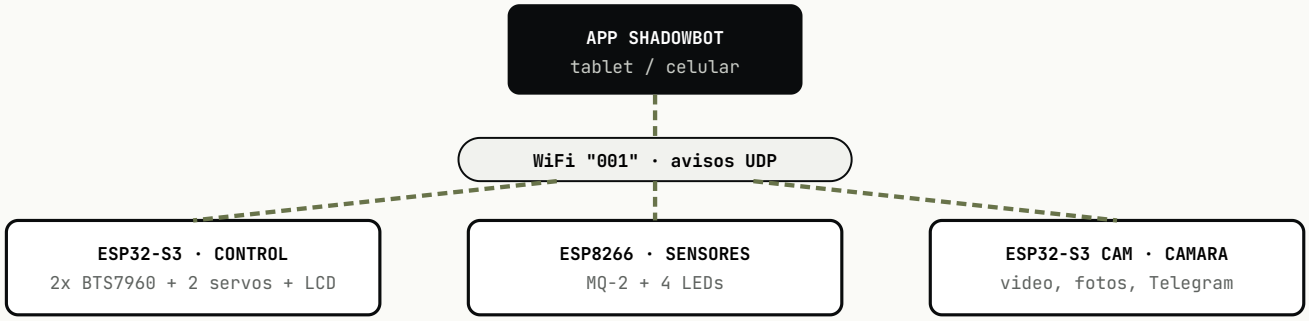
D ESP32-S3 CAM · SOLO ALIMENTACIÓN



PIN ESP32-S3 CAM	VA A	NOTA
5V	Fuente 5 V (o el USB-C)	la cámara + WiFi piden picos de corriente: fuente de 1 A o más
GND	GND de la fuente	si comparte la fuente con CONTROL, unir GND
Cámara	Conector FPC de la placa	cinta con los contactos hacia la placa, seguro bien cerrado

CÓDIGO: CODIGOS SIN GC9A01\CAMARA\CAMARA.ino · placa "ESP32S3 Dev Module" · **PSRAM: OPI PSRAM** · librería ArduinoJson. Al arrancar, el Monitor Serie (115200) dice "PSRAM: OK" o "PSRAM: NO encontrada"; la cámara funciona en los dos casos.

E SISTEMA COMPLETO · 3 PLACAS



SIN CABLES ENTRE PLACAS. Las 3 placas se hablan solo por WiFi (red "001"). Lo único que se puede compartir es la fuente de 5 V y su GND.